

A Real but Untradable Signal in a Liquid On-Chain Market: Learned Short-Horizon Prediction on Uniswap v3 WETH/USDC

Dennis Decoene

September 2026

Abstract

We ask whether a purpose-trained neural network can find a tradable short-horizon signal in the public swap history of the most liquid Uniswap v3 pool, WETH/USDC at the 0.05% fee tier. We ingest about 200 days of swap events directly from an archive node, build one-minute bars, label each bar with the return of a triple-barrier trade, and evaluate on chronological hold-out data. We find a signal, and it is not tradable. Under a walk-forward study with five folds, five seeds and purged, overlapping-label-aware inference, a model trained on the 30-minute pre-cost price move reaches a pooled out-of-sample correlation of +0.034 (95% interval +0.023 to +0.045, permutation $p = 0.001$), positive in every fold and stable across seeds. It weakens at 120 minutes (+0.021) and is indistinguishable from zero at 240 minutes. The signal is about a twelfth of the size needed: the model’s most confident 1% of bars average a gross move of +0.012% against a round-trip cost of 0.148%, and every trading rule we replay loses money on both the validation and the test window. An earlier model trained on net return reached a test correlation of 0.35, but that was cost prediction (−0.96 with the cost part of the label, +0.01 with the price move). Order-flow features and a larger network do not change the signal. The pipeline recovers planted signals in synthetic data (correlation 0.82 to 0.85) and finds nothing in a random walk. We also report a failed earlier attempt with a general-purpose language model, and a unit error in the signal path that produced a spurious −43% backtest.

1 Introduction

A common picture of machine-learning trading is that a model is given data, its weights adjust until a pattern emerges, and the pattern is then applied to unseen data to predict where the price goes next. This describes the mechanism correctly and leaves out three things. A model can only find a pattern that is in the data; adjusting weights cannot create one. A flexible model will fit noise in its training data, so only performance on data it has not seen separates a pattern from a fit. And a real pattern must still be large enough to pay for the cost of trading on it.

This paper documents one complete attempt to find such a pattern. It finds a real one, and shows that it is far too small to trade. The setting is favourable to a search: the data are public, complete and free of survivorship problems, since every swap is a log entry on chain, and the venue’s cost structure (a fixed fee, gas, and a price-impact curve determined by the pool’s liquidity) can be computed for each trade. The setting is unfavourable to finding an edge for the standard reason: the pool is heavily watched and arbitrated, and the inputs are visible to every participant.

The contributions are modest and practical.

1. A leakage-controlled pipeline from raw swap logs to a cost-aware backtest, with a planted-signal test showing it finds signals that exist.

2. A measured result on about 200 days of WETH/USDC data: a faint but statistically detectable direction signal at 30 minutes, with walk-forward evaluation, seed spread, block-bootstrap intervals and a permutation test, and the sweep records that support it. No trading rule is profitable.
3. A diagnostic, splitting each label into its price move and its cost, that shows a good-looking fit statistic under a net-return target was cost prediction and carried no information about direction.
4. An account of two failures along the way (Section 10 and Section 8) that are as instructive as the result.

We separate statistical from economic significance. Section 6.6 shows the signal is distinguishable from zero; Section 6.7 shows why it cannot pay for the cost of trading it.

2 Background

Market efficiency. The weak form of the efficient-market hypothesis [1] says that prices already reflect the information in past prices, so patterns in past prices and volumes should not predict returns after costs. The claim is about profit after costs, so a faint pattern that costs more to trade than it earns is consistent with it. The value of the experiment is in measuring how faint the pattern is, and in showing where a plausible-looking signal came from, not in overturning the hypothesis.

Uniswap v3. Uniswap v3 [2] is a constant-product market maker with concentrated liquidity. Each swap emits an event carrying the post-trade $\sqrt{P}$ as a Q64.96 fixed-point number (`sqrtPriceX96`), the current tick, the liquidity active at the price, and the signed token amounts. The WETH/USDC 0.05% pool charges 5 basis points per swap. A trader also pays gas and, on large trades, price impact that depends on the liquidity near the price. Liquidity providers face adverse selection against informed flow [3], which is one reason to expect short-horizon price moves to be hard to predict from the pool’s own history.

Triple-barrier labels. We follow the triple-barrier idea of López de Prado [4]: a label is the return of a trade entered at a bar and closed when the price first moves a set distance in either direction or when a time limit is reached.

3 Related work

Predictability from order flow and microstructure. Order-flow imbalance explains a substantial share of short-horizon price changes in limit order books [6], and deep networks trained on order-book data find features that hold across instruments [7]. Those results concern the best quotes of centralized venues at horizons of seconds. A concentrated-liquidity pool has no order book, its state is the last price and the liquidity at it, and we use one-minute bars and horizons of half an hour and longer, where costs are a larger share of the typical move. We test whether an analogous signal, built from signed swap flow, carries over.

Machine learning for return prediction. Gu, Kelly and Xiu [8] show that neural networks improve forecasts of equity returns, with gains concentrated in interactions among many characteristics. Our inputs are far fewer and all derived from one pool’s history, and the result is different: neither a larger network nor extra features moves the signal.

Overfitting and evaluation. A trading backtest that reports the best of many trials overstates skill [9, 10]. Our protocol was not fixed in advance: the five runs in Section 6 were chosen one after another in response to earlier results. We therefore report every run, none is dropped, and the robustness study (Section 5.5) uses a protocol set before it was run. We use chronological splits, a purge of one label length between train, validation and test, and resampling methods that respect overlapping labels (Section 5.5). For dependent data we use the moving-block bootstrap [11].

4 Data

Source. All swaps come from `eth_getLogs` calls for the pool’s `Swap` event against an archive endpoint, decoded to price, tick, in-range liquidity, amounts, and the block’s base fee. The dataset covers blocks 24,642,884 to 26,084,902, about 200 days, on Ethereum mainnet.

Ingestion. The ingester records which block intervals have been fetched as a set of disjoint intervals, fills gaps, and never requests a block twice. Writes are atomic and resumable, and pending results are flushed to disk before an error is raised. Blocks are fetched from newest to oldest, so any interruption leaves one continuous stretch anchored at the newest block (or at data already on disk), and the partial dataset is immediately usable. Training refuses data with holes.

Rate limiting. The provider is a free-tier endpoint, and its request rate sets the ingest speed. Fixed concurrency slots were replaced by a proactive requests-per-second limiter that learns a safe rate, backs off to the last known-good rate after an HTTP 429, raises its ceiling again when things go well, and persists what it learned. Empirically about 6.2 requests per second was the best rate: at or below it there were no 429 responses, and above it each throttle carried a wait that cost more than the extra request rate gained.

Measured ingest time. The ingest log spans 25 to 29 September 2026 (four calendar days) and records 44 runs, most of them stopped by hand while the limiter was being tuned. Summed over runs, the active time was about 28.4 hours for roughly 1.55 million blocks requested, a little more than the 1.44 million on disk because of restarts. Runs at a steady rate of about 6.5 requests per second in chunks of 10 blocks covered 1,600 to 1,800 blocks per minute, which puts a clean pass over the 200-day window at roughly 14 hours. The overall average was lower, about 900 blocks per minute, because of the slowest run, an overnight run on 28 September that averaged 340 blocks per minute over 15 hours. It shows repeated waits of 10 to 45 minutes between chunks, when the provider was throttling. Wall-clock time to add history is therefore set by the provider’s limits as much as by the request rate.

Limitation of the collection method. This is probably not the most efficient way to obtain the data. A provider with bulk export, a self-hosted node, or a public swap dataset would likely be faster. `eth_getLogs` over an archive endpoint was the approach known to work for free, running a node was not possible with the available hardware and budget, and there was no time pressure, so a slow and safe ingest was acceptable. We report it as a constraint, not as a recommended method.

5 Method

5.1 Bars and features

Swaps are aggregated into one-minute bars holding the last price, swap count, volume in USDC, signed volume (net buying of WETH), the largest single swap, and the last block base fee. Minutes with no swap carry the price and fee forward and are flagged as gaps.

Seven price features are computed from past bars: returns over 5, 15 and 30 minutes, the rolling standard deviation of one-minute returns, volume, the number of bars since the last swap, and the base fee in gwei. Four order-flow features were added after the first null: flow imbalance over 5, 15 and 30 minutes (net signed volume over total volume, in $[-1, 1]$) and the share of the last 15 minutes' volume taken by the largest single swap.

All features are shifted back one bar before pairing with labels. A label at bar t enters at the first swap at or after the bar's start, which can precede the bar's own close, so an unshifted feature would leak up to one bar of post-entry price action into the input. Features are standardized with statistics from the training split only.

5.2 Labels and costs

For each bar the label is the round-trip return of a long trade with a reference notional of \$1,000, entered at the first swap at or after the bar start. The trade exits when the price first moves by a band of two standard deviations of the pool's one-minute return, or after 30 minutes. Bars whose horizon extends beyond the ingested data are excluded rather than guessed.

Two returns are stored for every bar. The *gross* return is the price move alone. The *net* return subtracts (a) the pool fee on both legs, $2 \times 0.05\%$; (b) slippage on each leg, estimated as $\text{notional}/(2D)$ with D the virtual quote-token depth $L\sqrt{P}$ implied by the in-range liquidity, a concentrated-liquidity approximation and not exact tick-crossing math; and (c) gas, taken as 150,000 gas at the block base fee on each leg. The cost part of a label is $\text{net} - \text{gross}$. Round-trip cost averages about 0.13% of notional.

5.3 Model and split

The model is a fully connected network with ReLU units and one output, trained by full-batch Adam [5] on mean squared error against the standardized target, using MLX. The default has one hidden layer of 16 units, which is close to linear in practice. A second configuration has two hidden layers of 64 units, with early stopping on the validation loss (patience 50, best epoch restored). The stopping point uses the validation window, so validation figures are slightly optimistic and the test window is the judge.

Bars are split chronologically, 70% train, 15% validate, 15% test, with no shuffling. The test window is about 30 days (43,400 bars) and is used once per model.

5.4 Backtest and sweep

A backtest replays the exact fills the labels were computed from. Equity starts at \$10,000, a position is capped at 10% of equity, a daily loss limit of 2% applies, and capital is held for the full horizon. The *sweep* replays a set of decision rules over each of the validation and test windows: trade whenever the predicted return exceeds zero; trade whenever it exceeds a minimum edge of 0.13%; trade the model's top 1%, 5%, 10%, 25% or 50% of bars by predicted return, with the cutoff taken from the validation window; trade every bar; and, as a ceiling, an oracle that trades exactly the bars whose realized net return is positive. Three shuffled-prediction controls take the same number of trades at random positions.

Training records the feature set, label settings, target and network size with the model, and backtests and sweeps relabel using those recorded settings, so a run cannot be replayed under different settings by accident.

5.5 Robustness study

The single-split experiments leave open whether the correlation is distinguishable from zero, whether it depends on the seed or on one 30-day window, and whether a longer horizon changes it. A separate study answers these with the same features and the gross-return target.

Walk-forward folds. The last half of the history is cut into five consecutive test blocks. Each fold trains on all bars before its test block, except a validation stretch (the trailing 15% of that region) used for early stopping. One label length of bars is dropped between train and validation and between validation and test, so that overlapping labels cannot span the boundary. Every fold’s model therefore sees only the past of the bars it is judged on.

Seeds and horizons. Each fold is trained with five seeds. The study is run at label horizons of 30, 120 and 240 minutes, with the take-profit/stop-loss band widened with the square root of the horizon from two standard deviations at 30 minutes, since a band held fixed would be reached within minutes at any horizon.

Inference. Predictions are averaged over seeds within each fold and pooled over folds. Two quantities are then computed on the pooled out-of-sample series. A moving-block bootstrap [11], with blocks of one label length, gives a 95% interval for the gross correlation. A circular-shift permutation test, which shifts the predictions against the outcomes by at least one block and so keeps each series’ own dependence, gives a two-sided p-value for zero correlation. Both use 1,000 resamples. The spread of the correlation across seeds is reported separately. Both procedures are validated in the test suite on overlapping-noise series: the bootstrap interval is more than twice as wide as the naive $1/\sqrt{n}$ interval, and independent overlapping series are not flagged more than a fraction of the time.

Trading slices. For each fold the model’s top 1%, 5%, 10% and 25% of bars are selected using a cutoff taken from that fold’s own validation predictions. The study reports their mean gross return, mean cost and mean net return, with a block-bootstrap interval, against the same means over all bars. This is a per-trade measure and does not model capital or position limits, which the sweep does.

5.6 Validation of the pipeline

To test that the machinery can find a signal, we run it on synthetic swaps. On swaps whose price follows a hidden drift that flips sign every two hours, the pipeline recovers it with gross correlation 0.82 on validation and 0.85 on test. On a pure random walk the same run gives 0.03 and 0.02, while its correlation on the training data alone was 0.19, which is what fitting noise looks like. With a regime that tilts trade direction and, weakly, the price, and with lagged returns removed from the inputs, the model finds it from order flow (0.18 validation, 0.27 test) where a price-only model finds 0.07 and 0.13. On a combined pattern (drift sign is the product of a flow regime and a volatility regime), a 64,64 network without early stopping fit its training data almost perfectly (0.71) and did worse on unseen data (0.13) than the 16-unit model (0.19); with early stopping the sizes 16, 64,64 and 128,64 all reach about 0.21 on validation.

6 Results

Setting for all runs: WETH/USDC 0.05%, blocks 24,642,884 to 26,084,902, one-minute bars, 30-minute horizon, band of two standard deviations, chronological 70/15/15 split. Sweep records are in `docs/paper/results/`.

6.1 Experiment log

Run	Inputs and target	Gross corr. (test)	Cost corr. (test)	Outcome
1	7 price features, net	not measured	not measured	no edge
2	same model, diagnostics added	+0.01	-0.96	no edge; predicts cost
3	+ 4 order-flow features, net	+0.01	-0.96	no edge
4	same inputs, gross target	+0.04	+0.12	no edge
5	64,64 network, gross, early stop	+0.03	+0.02	no edge

Table 1: The five runs. Runs 1 and 2 use the same trained model; the second sweep adds the gross/cost decomposition and the shuffled control. Run 3’s validation gross correlation is +0.03, run 4’s is +0.04, run 5’s is +0.04.

6.2 The first model learned cost

Trained on net return, the first model reached a correlation of 0.35 with the net-return label on the test window, up from 0.25 on train and 0.23 on validate, with an MSE improvement over the mean baseline of about 7%, 6% and 12%. That looks like skill. Splitting each label into its price move and its cost changes the reading. On the test window the predictions correlate +0.01 with the gross return and -0.96 with the cost. The label mixes direction, which is hard to predict, with cost, which is easy: two of the inputs are gas price and volatility, which drive gas and slippage directly. The spread of the cost part is a fifth to two fifths of the gross part’s, yet it accounts for nearly all of the correlation. A model can fit a net-return label without knowing anything about direction, and the headline correlation hides it.

The model also never predicts a positive net return on the test window (mean -0.126%, maximum -0.046%), so the rule “trade when the prediction is above zero” takes no trades and the backtest ends at exactly the starting equity.

6.3 Trained on the price move

With the target changed to the gross return, the model no longer tracks cost (cost correlation falls from -0.96 to +0.12 and, with the larger network, to +0.02). It finds a faint direction signal: gross correlation +0.04 on validation and +0.03 to +0.04 on test, positive on every split. That is about 0.1% to 0.2% of the variance of the move.

6.4 More information and more capacity change nothing

Adding four order-flow features to the net-return model left the gross correlation at +0.01 on test (+0.03 on validation). Replacing the 16-unit network by a 64,64 network with early stopping, on the same inputs and the gross target, gave the same gross correlation (+0.04 validation, +0.03 test). Two models of very different capacity reading the same inputs agree, which points at the inputs as the limit and not at the model.

6.5 No trading rule made money

Table 2 shows the test window for the gross-trained 16-unit model (run 4) and the 64,64 model (run 5). Results on the validation window are the same in kind.

Rule	Run 4 (16 units)		Run 5 (64,64)	
	Trades	Return	Trades	Return
Predicted > 0	7,007	−22.6%	9,369	−20.8%
Predicted > 0.13%	0	0.0%	0	0.0%
Top 1% by prediction	298	−7.6%	118	−2.6%
Top 10% by prediction	2,466	−28.1%	1,541	−18.2%
Top 50% by prediction	5,257	−43.8%	5,166	−43.4%
Every bar	5,343	−44.5%	5,358	−44.5%
Oracle	15,924	+16.6%	15,927	+17.0%

Table 2: Test-window results, \$10,000 start, positions capped at 10% of equity. Returns are over the roughly 30-day window.

Trading every bar loses about \$0.83 per trade on average. The ranked slices lose about the same per trade at every cut (roughly \$0.8 to \$1.2), so the total loss shrinks only because fewer trades are taken. The model’s ranking has no per-trade value net of costs on this window. Against the shuffled control, run 4’s top-1% slice on the test window is worse than random (\$2.5 lost per trade against \$1.3, on 298 trades), and the other slices match it to within the noise of a few hundred trades. The minimum-edge rule takes zero to three trades because no predicted move approaches the round-trip cost. In the net model (run 2) the ranked slices beat shuffled predictions by about \$0.18 per trade, consistent with the model picking bars that are cheap to trade, which agrees with the finding above, and every rule still lost.

The oracle earns only \$0.10 to \$0.14 per trade and +15% to +20% per window even with perfect foresight of every trade, because typical 30-minute moves (about 0.17%) are small against a round-trip cost of about 0.13%.

6.6 Walk-forward study: the signal is real and small

The study of Section 5.5 covers the full 200 days: the last half of the history, 144,850 bars or about 100 days, is judged in five consecutive test folds, each by models trained only on earlier bars. It uses the 64,64 network on all eleven features and the gross target. The sweep record is <docs/paper/results/studies/20260929T192107Z.json>.

Horizon (min)	Band (std)	Gross corr. (pooled)	95% interval	p	Seeds (range)	Folds (range)
30	2.0	+0.034	+0.023, +0.045	0.001	+0.027, +0.032	+0.018, +0.064
120	4.0	+0.021	+0.002, +0.040	0.047	+0.011, +0.026	+0.008, +0.036
240	5.7	+0.014	−0.010, +0.036	0.340	−0.006, +0.024	−0.028, +0.053

Table 3: Pooled out-of-sample gross correlation. Intervals are moving-block bootstrap with blocks of one label length; p is a circular-shift permutation test, 1,000 resamples each, so 0.001 is the smallest value attainable. The effective number of independent labels is about 4,800, 1,200 and 600 at the three horizons. Seeds and folds give the range of the correlation over the five seeds (all folds pooled) and over the five folds (seeds averaged).

At 30 minutes the correlation is +0.034 with an interval that excludes zero, is positive in all five folds, and varies little across seeds (0.027 to 0.032). The single-window figures of the earlier runs, which looked like noise, fit this: with about 1,400 effective labels one window has a standard error near 0.03, and pooling 100 days cuts it to about 0.014. The signal weakens as

the horizon grows and disappears at 240 minutes, where the interval includes zero and one fold is negative. With three horizons tested, the 30-minute result survives a Bonferroni correction ($0.001 \times 3 < 0.05$) and the 120-minute result ($0.047 \times 3 = 0.14$) does not, so we treat only the 30-minute signal as established. The permutation test gives 0.001, its smallest attainable value, so the true p may be lower.

Horizon	Bars traded	n	Gross	Cost	Net	Net 95% interval
30	top 1%	2,204	+0.012%	0.148%	-0.136%	-0.159, -0.114
30	top 10%	17,050	+0.005%	0.123%	-0.119%	-0.126, -0.111
30	all	144,854	-0.005%	0.114%	-0.120%	
120	top 1%	1,965	+0.023%	0.154%	-0.131%	-0.184, -0.085
120	top 10%	16,416	+0.008%	0.124%	-0.117%	-0.137, -0.095
120	all	144,850	-0.001%	0.114%	-0.116%	
240	top 1%	1,054	+0.031%	0.281%	-0.250%	-0.454, -0.099
240	top 10%	14,484	+0.022%	0.131%	-0.109%	-0.148, -0.074
240	all	144,848	+0.006%	0.114%	-0.108%	

Table 4: Mean return per bar of the model’s top slices, on a \$1,000 round trip, pooled over the five folds. Each fold’s cutoff comes from its own validation predictions. Per-trade means, without capital or position limits.

Table 4 shows why this does not trade. The gross move of the selected bars rises with selectivity (at 30 minutes +0.001% for the top quarter, +0.005% for the top 10%, +0.012% for the top 1%), which is what a real ranking signal looks like, but the largest gross move, +0.012%, is a twelfth of the 0.148% cost of the same bars. Selected bars are also dearer to trade than average (cost 0.148% against 0.114%), because the model’s predictions correlate +0.13 with cost: it favours volatile, high-fee moments, where moves are larger and costs larger too. The net return of every slice is negative with an interval that excludes zero, and the top 1% is worse than trading every bar. At 240 minutes the correlation with cost rises to +0.35 and the top 1% pays 0.281%.

6.7 Why a small correlation cannot be traded here

Suppose predictions are approximately normal and a trade is taken at a prediction whose z-score is z . With correlation ρ between prediction and outcome, and σ the standard deviation of the move (0.17% on the test window), the expected gross move is about $\rho \sigma z$. A trade breaks even when this equals the round-trip cost $c \approx 0.13\%$, so

$$\rho z \approx \frac{c}{\sigma} \approx 0.76.$$

Trading only the top 1% of predictions (mean $z \approx 2.7$) still needs $\rho \approx 0.29$, against 0.034 observed, and the direct measurement in Table 4 agrees. The longer horizons do not rescue the strategy: a four-hour horizon needs $\rho \approx 0.27$ at $z = 1$, and the measured correlation there is +0.014.

The other side of the ledger is detection. Labels overlap: one 30-minute label per one-minute bar means the effective number of independent points is about 1/30 of the bar count. A single 30-day window gives roughly 1,400, a standard error near 0.03, so correlations under about 0.03 cannot be told from zero in one window, which is why the earlier single-split runs were inconclusive. The pooled walk-forward sample has about 4,800 effective labels and a standard error near 0.014, enough to see +0.034. Detecting a signal and being able to trade it are separate questions, and here the answers differ.

7 Discussion

The results fit market efficiency for a liquid, heavily arbitrated pool with public inputs, and the experiments add a decomposition of why the first attempt looked better than it was. Five points summarize it.

1. **A model can only find a pattern that is in the data.** On planted signals the pipeline finds them; on a random walk it finds nothing on unseen data.
2. **The pattern the first model learned was the wrong one.** Gas price and volatility set trading cost, cost is easy to predict, and a net-return target rewards learning it. The fit statistic looked good and said nothing about direction. It was found only by splitting the label.
3. **Unseen data is what separates a real pattern from a fitted one.** The chronological split is the instrument: 0.19 on training and 0.02 on unseen data for the random walk.
4. **A faint pattern that does not pay is the expected result.** The inputs were the last few minutes of price and volume in the most watched pool on chain. Easy patterns in public inputs are traded away, and what remains is small: a correlation of +0.034 that fades with the horizon and is gone at four hours.
5. **A real pattern must also clear cost.** Even perfect foresight earned only +15% to +20% per 30-day window here, so a weak signal is not enough.
6. **Statistical and economic significance differ.** The 30-minute signal has $p = 0.001$ and is a twelfth of the cost of trading it. Reporting only the first would suggest an edge; reporting only the second would hide that a signal exists.

8 Threats to validity

A unit error in the signal path. An earlier backtest lost 43% because the signal client returned the model’s standardized output (in standard deviations of the target) while the guard read it as a real return, so “predicted above zero” meant “above the average training return”. It was found because the result was suspicious, fixed in the signal client, and covered by a regression test. Every result produced before the fix is invalid and none is reported here.

Other threats.

- *One pool, one period.* The walk-forward study spans about 100 days of test data in five folds, which addresses the single 30-day window, but all of it is one pool in one stretch of 2026. The fold correlations at 30 minutes range from +0.018 to +0.064, so regime dependence is real though its sign did not change.
- *Simplified costs.* Slippage uses a depth approximation, not tick-crossing math, and gas uses the base fee at 150,000 gas per swap, ignoring priority fees and aggregator routing. Real costs may be higher. The gross-target diagnostics do not depend on the cost model.
- *Capital held for the full horizon* in the backtest, which understates capital efficiency when exits are early.
- *Validation window used for early stopping.* Validation figures for run 5 are slightly optimistic; the test window is unaffected.

- *Multiple comparisons and design selection.* Three horizons are tested, and we correct for that (Section 6.6). The features, the target and the network sizes were chosen after seeing the earlier runs, whose test windows overlap the folds of the study, so the study is a robustness check of a fixed design, not a fully blind test. The permutation p -value is bounded below by 0.001 at 1,000 resamples.
- *Gaps in the data.* Ingestion tracks fetched intervals and training refuses holes, but the archive endpoint itself is a single source.
- *Cost model.* The slice costs in Table 4 use the same simplified cost model as the labels, and the model’s mild preference for costly bars means a better cost model would change the net figures.

9 Limitations and future work

The following were not tested and are not claimed: horizons beyond four hours, other pools and less liquid ones, other bar sizes, model families other than a small feed-forward network, data outside these 200 days, a control without gas price and volatility, and information outside the swap logs such as other venues or the mempool. The signal’s decay with horizon is measured at three horizons only. Shorter horizons, where a signal of this size is a larger share of the typical move, are the natural next step, though they face the same fixed cost per trade. Liquidity-provision fee income, which turns the question from prediction into carry, is also untested. Cross-market transfer and a less liquid pool are the most obvious next experiments.

10 An earlier attempt

The project began, on 23 September 2026, as an order-book bot for Solana using Phoenix order-book snapshots, Jupiter for execution, and a safety guard for sizing and daily loss limits. The decision model was a general-purpose typed-decision text classifier served locally. It was asked whether now was a good moment to trade, given one text line with best bid, best ask and depth counts, and it traded when its confidence passed a threshold.

The backtest used historical prices from a public price API. These carry no order-book data, so the model was fed a synthetic bid/ask ladder generated from the spot price plus a fixed cost in basis points, a deterministic function of price. The result was a win rate of about 23%, worse than buy-and-hold. These figures are recalled: the result files did not survive the rewrite, and we did not re-run the old code.

The cause is clear enough without them. The model had never seen financial time series, had no learned mapping from these inputs to price behaviour, and answered “trade” almost regardless of input. The inputs carried no predictive information beyond the price itself, and a single snapshot has no history. The restart on 24 September applied the rules this paper follows: a purpose-trained model, the venue’s own on-chain data, net-of-cost labels, and a chronological split with a test window used once.

11 Conclusion

On about 200 days of WETH/USDC 0.05% swaps, a small neural network trained on the pre-cost price move finds a faint direction signal at a 30-minute horizon: a pooled out-of-sample correlation of +0.034 (95% interval +0.023 to +0.045), positive in every walk-forward fold and stable across seeds. It fades at 120 minutes and is absent at 240. It is about a twelfth of the size needed to cover the cost of trading it, and every trading rule loses. What looked like much stronger skill under a net-return target was cost prediction. Order flow and network size do not

move the signal, and the pipeline recovers planted signals, so the result concerns the market and its inputs and not the machinery. The finding is a measurement of how small the predictable part of a liquid on-chain market is, and that it does not pay for trading it.

References

- [1] E. F. Fama. Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2):383–417, 1970.
- [2] H. Adams, N. Zinsmeister, M. Salem, R. Keefer, and D. Robinson. Uniswap v3 core. Technical report, Uniswap Labs, 2021.
- [3] J. Millionis, C. C. Moallemi, T. Roughgarden, and A. L. Zhang. Automated market making and loss-versus-rebalancing. arXiv:2208.06046, 2022.
- [4] M. López de Prado. *Advances in Financial Machine Learning*. Wiley, 2018.
- [5] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- [6] R. Cont, A. Kukanov, and S. Stoikov. The price impact of order book events. *Journal of Financial Econometrics*, 12(1):47–88, 2014.
- [7] J. Sirignano and R. Cont. Universal features of price formation in financial markets: perspectives from deep learning. *Quantitative Finance*, 19(9):1449–1459, 2019.
- [8] S. Gu, B. Kelly, and D. Xiu. Empirical asset pricing via machine learning. *The Review of Financial Studies*, 33(5):2223–2273, 2020.
- [9] D. H. Bailey, J. M. Borwein, M. López de Prado, and Q. J. Zhu. Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance. *Notices of the AMS*, 61(5):458–471, 2014.
- [10] H. White. A reality check for data snooping. *Econometrica*, 68(5):1097–1126, 2000.
- [11] H. R. Künsch. The jackknife and the bootstrap for general stationary observations. *The Annals of Statistics*, 17(3):1217–1241, 1989.